

Wednesday 5:46 PM



lab_one.pdf

PDF

Here is my lab, run me though what we doing, what language, what is the task, how can i go about it



How do the files look from a glance, also why is there a
claude.md file?

No, Ari — those warnings are **not critical**. Your files are still being added normally.

They mean:

- **LF** = Linux/macOS line endings
- **CRLF** = Windows line endings
- Git is warning that it may convert the files to Windows-style line endings next time it updates them

This does **not** damage the code or affect the application.

Continue with:

PowerShell

Check that `.env` is absent and the correct files are staged. Then commit:

PowerShell

You do not need to fix the LF/CRLF warnings for this lab. They are Git being dramatic about invisible characters.



whats next

Worked for 59s >

Ari, next we connect Prisma to Next.js and build the first real feature: **create a task and read it back from SQLite**.

Before that, confirm the database commit exists:

```
git status
git log --oneline -2
PowerShell
```

You want:

```
nothing to commit, working tree clean
```

and two commits in the log. If the changes are still staged, commit them first:

```
git commit -m "Add SQLite task schema and initial migration"
git push
PowerShell
```

Step 1 — Create the Prisma connection

Inside `src`, create a folder named:

```
lib
```

Then create:

```
src/lib/prisma.ts
```

Paste:

```
import { PrismaBetterSqlite3 } from "@prisma/adapter-better-sqlite3";
import { PrismaClient } from "@generated/prisma/client";

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

const adapter = new PrismaBetterSqlite3({
  url: process.env.DATABASE_URL ?? "file:./dev.db",
});

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    adapter,
  });

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

This creates one reusable database connection. The global check prevents Next.js development reloads from repeatedly creating new Prisma clients. Current Prisma's SQLite setup uses the `better-sqlite3` adapter when constructing `PrismaClient`.  Prisma +1

Step 2 — Create the task action

Create:

```
src/app/actions.ts
```

Paste:

```
"use server";

import { revalidatePath } from "next/cache";
import { prisma } from "@lib/prisma";

export async function createTask(formData: FormData) {
  const title = formData.get("title")?.toString().trim();
  const description = formData.get("description")?.toString().trim();
```

```

const topic = formData.get("topic")?.toString().trim();
const dueDateValue = formData.get("dueDate")?.toString();

if (!title || !description || !topic || !dueDateValue) {
  throw new Error("All task fields are required.");
}

const dueDate = new Date(dueDateValue);

if (Number.isNaN(dueDate.getTime())) {
  throw new Error("The due date is invalid.");
}

await prisma.task.create({
  data: {
    title,
    description,
    topic,
    dueDate,
  },
});

revalidatePath("/");
}

```

A Server Action runs on the server, so the form can safely write directly to SQLite without creating a separate API endpoint.  Next.js +1

Step 3 — Replace the landing page

Open:

src/app/page.tsx

Replace everything with:

```

import { createTask } from "../actions";
import { prisma } from "@lib/prisma";

export default async function Home() {
  const tasks = await prisma.task.findMany({
    where: {
      archivedAt: null,
    },
    orderBy: {
      createdAt: "desc",
    },
  });
}

```



```

return (
  <main>
    <h1>Todo Application</h1>

    <section>
      <h2>Create a task</h2>

      <form action={createTask}>
        <div>
          <label htmlFor="title">Title</label>
          <input id="title" name="title" type="text" required />
        </div>

        <div>
          <label htmlFor="description">Description</label>
          <textarea id="description" name="description" required />
        </div>

        <div>
          <label htmlFor="dueDate">Due date</label>
          <input
            id="dueDate"
            name="dueDate"
            type="datetime-local"
            required
          />
        </div>

        <div>
          <label htmlFor="topic">Topic</label>
          <input id="topic" name="topic" type="text" required />
        </div>

        <button type="submit">Create task</button>
      </form>
    </section>

    <section>
      <h2>Active tasks</h2>

      {tasks.length === 0 ? (
        <p>No tasks have been created.</p>
      ) : (
        <ul>
          {tasks.map((task) => (
            <li key={task.id}>
              <h3>{task.title}</h3>
              <p>{task.description}</p>
              <p>Topic: {task.topic}</p>
              <p>Status: {task.status}</p>
              <p>Due: {task.dueDate.toLocaleString()}</p>
            </li>
          ))}
        </ul>
      )}
    </section>
  </main>
)

```

```

        </li>
      )}
    </ul>
  )}
</section>
</main>
);
}

```

Do not worry about appearance yet. The target is functionality:

Form submission → SQLite row created → task appears on page

Step 4 — Run it

PowerShell

Open:

`http://localhost:3000`

Create a test task using all four fields.

Then:

1. Refresh the browser.
2. Stop the server with `Ctrl + C`.
3. Start it again with `npm run dev`.
4. Confirm the task is still present.

That proves persistence—the heart of the lab rather than decorative button wizardry.

Do **not commit yet**. First confirm that the form loads and a task can be created; paste any error exactly as shown.

    Sources

Branched from [Branch · Full-stack Todo App](#)

```
>> git log --oneline -2
```

On branch main

Your branch is ahead of 'origin/main' by 1 commit.

(use "git push" to publish your local commits)

nothing to commit, working tree clean

a165523 (HEAD -> main) Added SQLite task schema and initial migration

f28eded (origin/main) Initial commit from Create Next App

Perfect, Ari. Your repository is in a clean state:

- a165523 is your second commit.
- No uncommitted changes remain.
- It exists locally but has **not yet been uploaded to GitHub**.
- origin/main still points to the initial commit.

Run:

PowerShell

Then confirm:

PowerShell

It should say:

```
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

That gives you two coherent commits towards the required minimum of six.

lab_one

Next: connect Prisma to the application

Create this file:

```
src/lib/prisma.ts
```

Paste:

```
import { PrismaBetterSqlite3 } from "@prisma/adapters-better-sqlite3";
import { PrismaClient } from "@generated/prisma/client";
```

```
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

const adapter = new PrismaBetterSqlite3({
  url: process.env.DATABASE_URL ?? "file:./dev.db",
});

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    adapter,
  });

TypeScript

globalForPrisma.prisma = prisma;
}
```

Then run:

PowerShell

```
npm run dev
```

At this stage, the landing page should still load without errors. Do not commit this file alone yet—we will combine it with task creation and listing as the next coherent commit.

    Sources

ChatGPT can make mistakes. Check important info.